27. SQL Ustalığı: Karmaşık Sorgular, İndeksleme ve AI Query Optimizer

EXPLAIN ANALYZE Okuyarak 10 Saniyelik Sorguları 5 Milisaniyeye Düşürmek
Bölüm 5 • Zorluk Derecesi: Orta / İleri Seviye • Tahmini Okuma Süresi: 13 dk

---

> 🎙️ **NotebookLM & Audio Overview Podcast Notları**
>
> **Podcast Tartışma Sorusu:**
> *"Veritabanına indeks eklemek sorguları 1000 kat hızlandırıyorsa, neden tüm kolonlara indeks eklemiyoruz? Gizli bedel nedir?"*
>
> **Bölümden Çıkarılacak 3 Altın Cümle:**
> - 📌 İndeks eklemek okuma (SELECT) hızını katbekat artırırken, her yazma (INSERT/UPDATE) işleminde ek CPU ve disk yükü getirir. - 📌 `EXPLAIN ANALYZE` çıktısındaki 'Seq Scan' (Ardışık Tarama) milyonlarca satırda sunucuyu boğan en büyük tehlikedir. - 📌 AI modelleri yavaş SQL sorgularını ve EXPLAIN planlarını analiz ederek en ideal B-Tree veya Composite indeksleri önerebilir.

📖 1. Giriş Hikayesi & Günlük Hayat Analojisi

1000 sayfalık kalın bir tıp ansiklopedisini düşünün.

Kitabın en arkasında alfabetik bir **Fihrist / Dizin (İndeks)** vardır. "Kalp krizi" kelimesini arıyorsanız, fihristten "K" harfine bakar ve doğrudan sayfa 412'yi açarsınız (İndeks Taraması - Index Scan). 5 saniye sürer!

Eğer o fihrist olmasaydı ne yapardınız? Sayfa 1'den başlayıp sayfa 1000'e kadar tüm sayfaları tek tek okumak zorunda kalırdınız! (Ardışık Tarama - Sequential Scan). 3 saat sürerdi!

Ama dikkat: Ansiklopediye yeni bir sayfa eklendiğinde (INSERT), arkadaki fihristi de baştan sona silip yeniden alfabetik düzenlemek gerekir!

İşte bu yüzden her kolona indeks konmaz; sadece sık aranan kritik kolonlara konur.

🧠 2. AI Çağında Yeni Zihniyet Modeli

SQL İndeksleme Mantığı (B-Tree):

```
Sorgu: SELECT * FROM users WHERE email = 'ali@test.com';

SENARYO A: İndeks Yok (Sequential Scan)
[Satır 1] ──> [Satır 2] ──> ... ──> [Satır 5.000.000] (Tüm disk okunur, 8 saniye sürer!)

SENARYO B: B-Tree İndeksi Var (Index Scan)
                  [ Root: L-Z ]
                 /             \
           [ A-K ]               [ M-Z ]
           /
   [ 'ali@test.com' ──> Satır Adresi: Block 42, Offset 8 ] (Sadece 3 adımda bulunur, 2 milisaniye!)
```

🤖 3. AI Prompt Laboratuvarı (Kopyala-Yapıştır Hazır Prompt)

Aşağıdaki prompt şablonunu Cursor, Claude Code, ChatGPT veya Copilot ortamınızda doğrudan kullanabilirsiniz:

```
# ROL: PostgreSQL Veritabanı Performans Tuning Uzmanı
Aşağıdaki SQL sorgusu 2 milyon satırlık `orders` tablosunda 7.5 saniye sürüyor.

## YAVAŞ SORGU:
```sql
SELECT id, user_id, total, created_at
FROM orders
WHERE status = 'PAID' AND created_at >= '2026-01-01'
ORDER BY created_at DESC
LIMIT 20;
```

EXPLAIN ANALYZE ÇIKTISI:

```
Seq Scan on orders (cost=0.00..45210.00 rows=152000 width=32) (actual time=12.50..7450.21 ms)
```

GÖREV:

1. Bu sorgunun neden bu kadar yavaş olduğunu açıkla.
```

2. Bu sorguyu 5 milisaniyenin altına indirecek tam Composite Index (Bileşik İndeks) SQL komutunu yaz.
3. İndeks sıralamasının (status, created_at) neden önemli olduğunu açıkla.

```
---

## 💻 4. Kod İncelemesi: Acemi / Halüsinasyon vs. Üretim Kalitesi

#### ❌ Acemi / Kontrolsüz AI Çıktısı (Riskli):
```sql
-- Kötü Çözüm: Ayrı ayrı iki bağımsız indeks oluşturmak (Bitmap Index Scan karmaşası)
CREATE INDEX idx_orders_status ON orders(status);
CREATE INDEX idx_orders_created ON orders(created_at);
-- Veritabanı iki indeksi birleştirmeye çalışırken gereksiz CPU yakar!
```

✅ Kıdemli Mühendis Tarafından İyileştirilmiş Sürüm (Güvenli & Temiz):

```sql
-- Mükemmel Çözüm: Sorgu Filtresine ve Sıralamasına Birebir Uyumlu Composite Index
-- Sıralama: Önce eşitlik (equality: status), sonra aralık ve sıralama (range: created_at)
CREATE INDEX idx_orders_status_created
ON orders (status, created_at DESC);

-- İsteğe Bağlı: Index-Only Scan için kapsayıcı indeks (Covering Index)
-- Tabloya hiç gitmeden sadece indeksten yanıt döner:
CREATE INDEX idx_orders_optimized
ON orders (status, created_at DESC)
INCLUDE (user_id, total);
```

💡 **Kod Analizi:** `Composite Index` oluşturulurken altın kural: Önce `=` ile aranan eşitlik kolonu (`status`), ardından `>=` veya `ORDER BY` ile kullanılan kolondur (`created_at DESC`). `INCLUDE` eki ise tabloya (heap) gitme ihtiyacını tamamen ortadan kaldırır (Index-Only Scan).

⚠️ 5. Junior Tuzakları & Kıdemli Bakış Açısı

- **Tuzak:** Tablodaki her kolona ezbere indeks koyup INSERT işlemlerini süründürmek.
  - **Çözüm:** Sadece WHERE, JOIN ve ORDER BY koşullarında sıkça kullanılan kolonları indeksleyin.
- **Tuzak:** İndekslenen kolon üzerinde fonksiyon çalıştırmak (Örn: `WHERE LOWER(email) = 'ali'`).
  - **Çözüm:** Fonksiyonlar standart B-Tree indeksini bozar; ya veriyi küçük harf kaydedin ya da Expression Index (`CREATE INDEX ON users(LOWER(email))`) oluşturun.
- **Tuzak:** Production ortamında kilitlenme (lock) yaratan indeks komutları çalıştırmak.
  - **Çözüm:** Canlı veritabanında indeks eklerken daima `CREATE INDEX CONCURRENTLY` kullanın.

```

🎬 6. Udemy Video Dersi Taslağı & Slayt Maddeleri

📽️ Ders Başlığı: *Ders 27: SQL Ustalığı - EXPLAIN ANALYZE ve İndeks Optimizasyonu*

- 🖥️ **Slayt 1:** Veritabanı fihristi nedir? B-Tree indeks nasıl çalışır?
- 🖥️ **Slayt 2:** EXPLAIN ANALYZE okuma rehberi: Seq Scan vs Index Scan.
- 🖥️ **Slayt 3:** Bileşik İndekslerde (Composite Index) kolon sırası kuralı.
- 🖥️ **Slayt 4:** Production güvenliği: CREATE INDEX CONCURRENTLY.

Canlı Kodlama / Ekran Kaydı Senaryosu:
PostgreSQL'de 1.000.000 satırlık mock sipariş tablosu oluşturun. İndeks olmadan sorguyu çalıştırıp 8 saniye sürdüğünü ve 'Seq Scan' verdiğini gösterin. Ardından composite index ekleyip sürenin 3 milisaniyeye düştüğünü canlı kanıtlayın.

🎯 7. Günün Uygulamalı Görevi & Mini Sınav

🛠️ Günün Görevi:

Projenizdeki en yavaş çalışan veritabanı sorgusunu tespit edin. Başına `EXPLAIN ANALYZE` ekleyerek 'Seq Scan' olup olmadığını kontrol edin ve gerekiyorsa indeks ekleyin.

🧠 Bölüm Sonu Kontrol Testi:

Soru 1: Bir SQL sorgusunun yürütme planında 'Seq Scan' (Sequential Scan) görülmesi ne anlama gelir? - A) Sorgunun ışık hızında çalıştığını - B) Veritabanının uygun bir indeks bulamadığı için tablodaki tüm satırları baştan sona tek tek taramak zorunda kaldığını - C) Sadece Python'da hata olduğunu - D) Tablonun tamamen boş olduğunu > **Doğru Yanıt:** B > **Açıklama:** Seq Scan, aranan koşula uygun indeks bulunmadığında tablodaki tüm satırların diskten taranmasıdır; büyük tablolarda çok ciddi performans kaybıdır.

Soru 2: Canlı (production) bir PostgreSQL veritabanında tabloyu kilitlemeden ve yazma işlemlerini durdurmadan indeks eklemek için hangi anahtar kelime kullanılır? - A) FAST - B) CONCURRENTLY - C) ASYNC - D) NOWAIT > **Doğru Yanıt:** B > **Açıklama:** `CREATE INDEX CONCURRENTLY` komutu, tablo üzerinde okuma ve yazma kilitlerini engeller; arka planda güvenle indeks inşa edilmesini sağlar.